MCP

(Model Context Protocol)

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. What is MCP ?

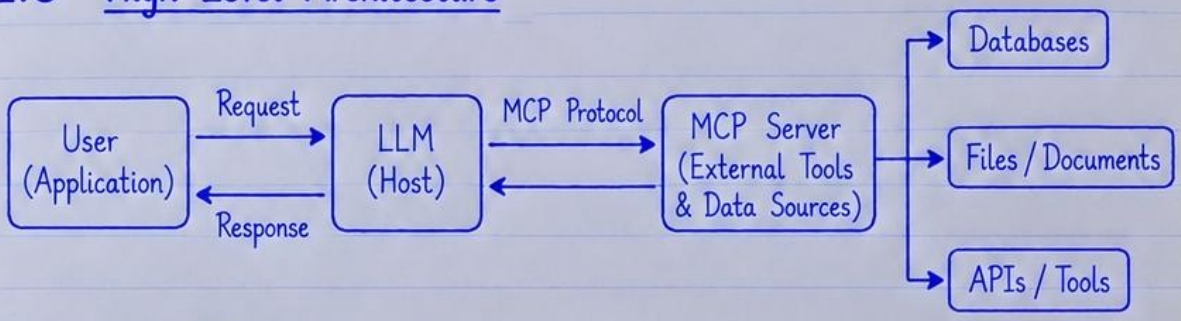
1.1 Introduction

- MCP stands for Model Context Protocol.
- It is an open standard protocol designed to connect Large Language Models (LLMs) with external tools, data sources and applications in a structured and secure way.
- MCP acts as a universal interface between LLMs and the real world.
- It helps LLMs to access live data, execute actions, and use tools without custom integrations for every service.

1.2 Why MCP ?

- LLMs are powerful, but they have limited knowledge (training data cutoff)
- They cannot directly access real-time data, files, databases or external tools.
- MCP provides a standard way to connect LLMs with external systems.
- It simplifies integration and makes LLM applications more capable and useful.

1.3 High Level Architecture



1.4 Key Points

- MCP is an open and standardized protocol.
- It enables LLMs to interact with external systems.
- It supports multiple types of tools and resources.
- It improves the capabilities and reliability of LLM applications.
- It is designed with security, scalability and flexibility in mind.

1.5 Common Use Cases

- Accessing real-time data (e.g. weather, stock prices, news)
- Reading and writing files
- Querying databases
- Calling external APIs (e.g. GitHub, Notion, Slack, etc.)
- Performing automated actions (e.g. send email, create ticket, etc.)
- Building AI agents with tool use capabilities

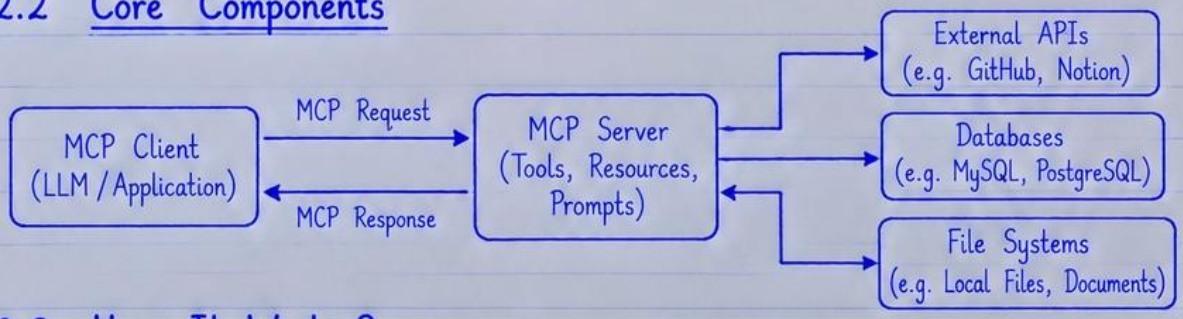
Rahul Yadav

2. Core Concepts & Terminology

2.1 Important Terms

Term	Description
MCP	Model Context Protocol. An open standard protocol for connecting LLMs with external tools and data sources.
MCP Server	A server that exposes tools, resources and prompts to LLMs using the MCP protocol.
MCP Client	A client (usually an LLM host or application) that connects to an MCP server to use its capabilities.
Tools	Functions that perform specific actions (e.g. search, read file, call API).
Resources	Data sources like files, databases or external content that can be accessed.
Prompts	Predefined prompts or templates provided by the server.
Host	The application that runs the LLM and acts as an MCP client.

2.2 Core Components



2.3 How It Works ?

1. MCP client sends a request to the server.
2. Server understands the request and selects the appropriate tool or resource.
3. Server executes the action (e.g. read file, call API, query database).
4. Result is returned to the client in a standard MCP format.
5. LLM uses the result to generate a response for the user.

2.4 Types of Capabilities

Capability Type	Description
Tools	Executable functions that perform actions (e.g. search, create file).
Resources	Readable data sources (e.g. documents, database records).
Prompts	Reusable prompt templates for specific tasks.

2.5 Key Benefits (Recap)

- Standardized way to connect LLMs with external systems.
- Works with multiple tools and data sources.
- Secure and controlled access.
- Makes LLM applications more powerful and extensible.

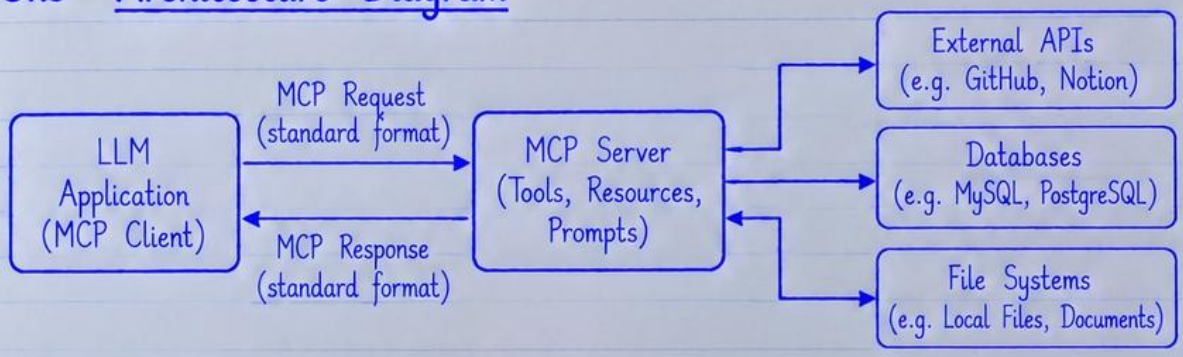
Rahul Yadav

3. MCP Architecture (High Level)

3.1 Overview

- MCP follows a client-server architecture.
- LLM application (MCP client) sends a request to an MCP server.
- MCP server communicates with external tools, data sources or APIs.
- Results are returned to the client in a standard format.
- The architecture is modular and can support multiple servers and tools.

3.2 Architecture Diagram



3.3 Components Explained

Component	Description
MCP Client	An application or LLM host that sends requests and uses the response from the server.
MCP Server	Exposes tools, resources and prompts using MCP protocol.
Tools	Functions that perform actions (e.g. read file, call API, search).
Resources	Data sources like files, databases or external content.
Prompts	Reusable prompt templates provided by the server.
External Systems	APIs, databases, file systems or any third-party service.

3.4 Request - Response Flow

1. Client sends a request with the desired action and parameters.
2. Server validates the request and checks available tools or resources.
3. Server executes the action (e.g. read file, query database, call API).
4. Result is returned in a standard MCP response format.
5. LLM uses the response to generate a final answer for the user.

3.5 Key Characteristics

- Standardized protocol.
- Works with multiple LLMs and tools.
- Modular and extensible.
- Supports real-time data and actions.
- Secure and controlled access to external systems.
- Easy to integrate in existing applications.

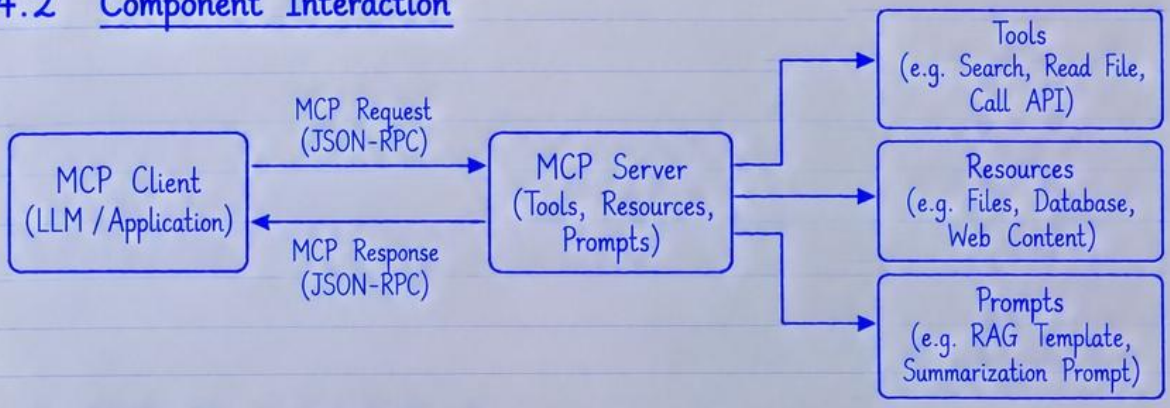
Rahul Yadav

4. MCP Components (Client, Server, Tools)

4.1 Main Components

Component	Description
MCP Client	Application or LLM host that sends requests to an MCP server and uses the response.
MCP Server	Exposes tools, resources and prompts using MCP protocol.
Tools	Functions that perform specific actions (e.g. read file, call API).
Resources	Data sources like files, databases or external content.
Prompts	Reusable prompt templates provided by the server.

4.2 Component Interaction



4.3 MCP Client (LLM Host)

- It can be an LLM application like Claude, ChatGPT, or a custom app.
- Sends requests to the MCP server using standard protocol.
- Receives structured responses from the server.
- Uses the response to perform actions and generate final output.

4.4 MCP Server

- Acts as a bridge between LLMs and external systems.
- Exposes tools, resources and prompts.
- Handles authentication, validation and execution.
- Can connect to multiple data sources and services.

4.5 Tools

- Executable functions that perform real actions.
- Examples: search web, read file, call API, query database, send email.
- Each tool has a name, description and input parameters.
- Server executes the tool and returns the result in standard format.

4.6 Resources

- Provide access to external data like files, databases, or APIs.
- Used to retrieve context that can be passed to the LLM.
- Examples: document files, knowledge base, database records, web content.

4.7 Prompts

- Predefined and reusable prompt templates.
- Helps in consistent and effective interactions.
- Examples: summarization prompt, RAG prompt, code generation prompt.

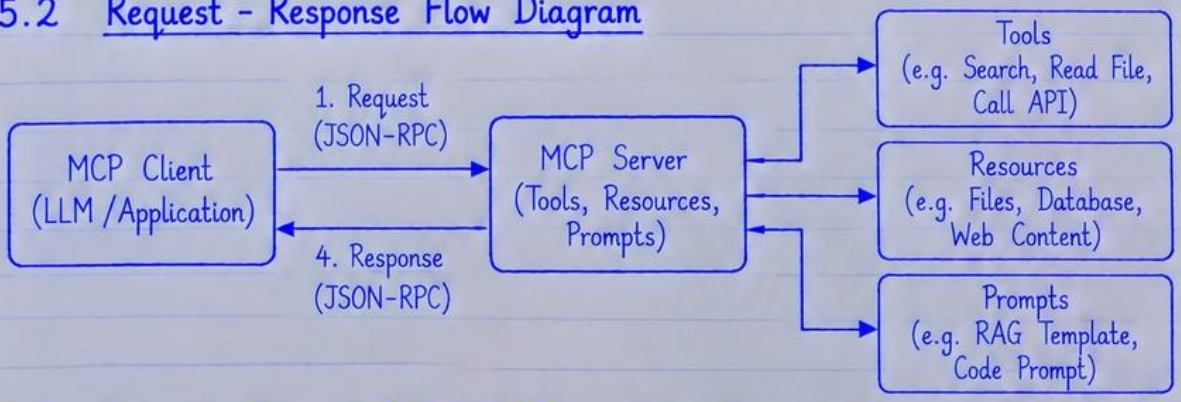
Rahul Yadav

5. MCP Communication & Data Flow

5.1 Communication Flow

- MCP uses a client-server model with a standard protocol.
- Client sends a request in a structured format (usually (JSON-RPC).
- Server processes the request, interacts with tools/resources and returns a response.
- The communication can be over stdio, HTTP or other supported transports.
- It enables secure and standardized communication between LLMs and external systems.

5.2 Request - Response Flow Diagram



5.3 Example Request (JSON-RPC)

Request from Client

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "search_web",
    "arguments": {
      "query": "Latest AI news"
    }
  }
}
```

5.4 Example Response (JSON-RPC)

Response from Server

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Here are the latest AI news..."
      }
    ]
  }
}
```

5.5 Supported Transports

Transport	Description	Use Case
stdio	Communication via standard input/output.	Local development, CLI tools.
HTTP	Communication over HTTP.	Remote servers, web services.
SSE (Server Sent Events)	Real-time streaming communication.	Real-time responses, streaming data.

5.6 Key Points

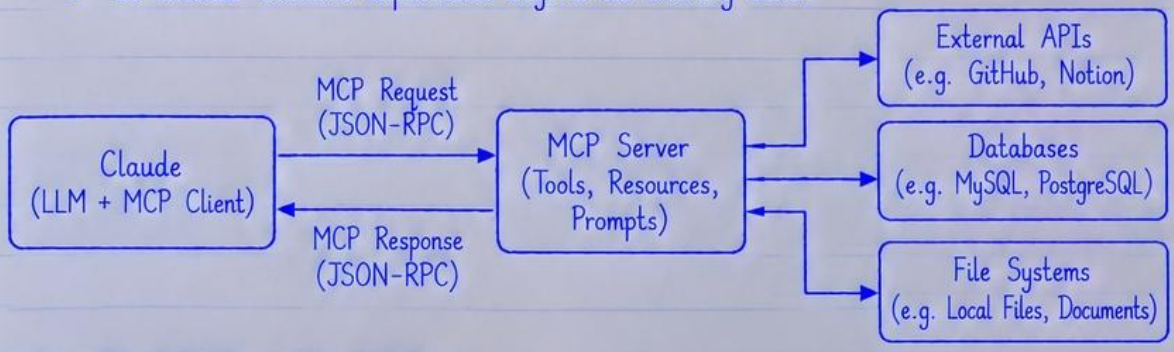
- Uses JSON-RPC 2.0 as the standard message format.
- Supports multiple transport mechanisms (stdio, HTTP, SSE).
- Provides a consistent request-response structure.
- Can handle tools, resources and prompts.
- Enables secure, extensible and real-time communication.

Rahul Yadav

6. Using Popular Tools with MCP

6.1 Claude with MCP

- Claude (Anthropic) can act as an MCP client.
- It can connect to MCP servers to access tools, data sources and custom functions.
- MCP allows Claude to perform real-time actions like searching data, reading files, querying databases, etc.
- It extends Claude's capabilities beyond its training data.



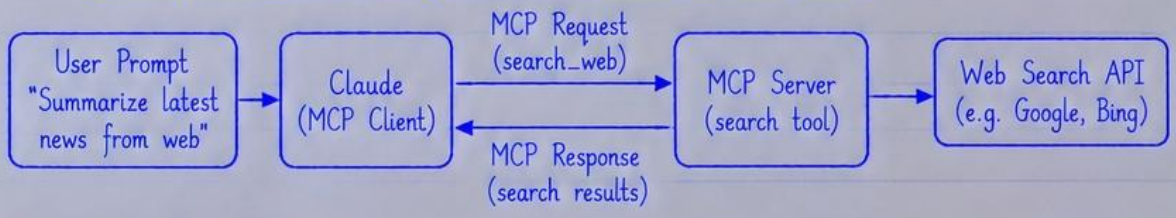
6.2 ChatGPT with MCP

- ChatGPT can use MCP through frameworks or custom integrations.
- It can connect to MCP servers to access tools and live data.
- Allows ChatGPT to perform actions like search, file read/write, API calls, etc.
- Makes ChatGPT more useful for real-world applications.

6.3 Other Tools and Integrations

Tool / Platform	How it Uses MCP	Example Use Case
Claude	Native MCP client support	Search files, call APIs, query DB
ChatGPT	Integration via frameworks	Use external tools and data sources
LangChain	Can use MCP servers as tools	Build agents with external tools
LlamaIndex	Integrate MCP for data access	Query documents, databases
Custom Applications	Implement MCP client	Connect to internal tools and APIs

6.4 Example: Claude Using an MCP Tool (Conceptual)



6.5 Key Benefits

- Works with popular LLMs like Claude and ChatGPT.
- Provides real-time data and tool access.
- Extends LLM capabilities.
- Standardized and reusable integration.
- Can be used in custom applications and frameworks.

Rahul Yadav

7. MCP Use Cases & Real-World Applications

7.1 Common Use Cases

- Connect AI assistants with external tools and data.
- Automate workflows across multiple services.
- Build custom integrations for enterprise applications.
- Enable agents to perform complex tasks.

7.2 Real-World Examples

- ChatGPT + GitHub: Read code, create PRs, manage repos.
- Claude + Notion: Fetch and update notes.
- Cursor + MCP: Use MCP servers for development tools.
- Enterprise: Connect AI with internal databases and APIs.

7.3 Industry Applications

- Software Development: Code search, debugging, file operations.
- Business Analytics: Fetch real-time data and generate reports.
- Healthcare: Access patient data and tools.
- Finance: Analyze transactions and risk data.

7.4 How MCP Helps?

- ✓ Bridges AI with real-world tools.
- ✓ Increases productivity and automation.
- ✓ Provides secure and controlled access.
- ✓ Supports extensibility and customization.

7.5 Comparison: MCP vs Other Approaches

Feature	MCP	Function Calling	Plugins/Extensions
Standardization	Yes	Limited	No
Flexibility	High	Medium	High
Tool Access	Unified	Per Model	Depends
Security	Strong	Medium	Varies

7.6 Key Takeaways

- ★ MCP is a powerful open standard for connecting AI with tools.
- ★ It simplifies integrations and improves agent capabilities.
- ★ It's built for a multi-tool, multi-server ecosystem.

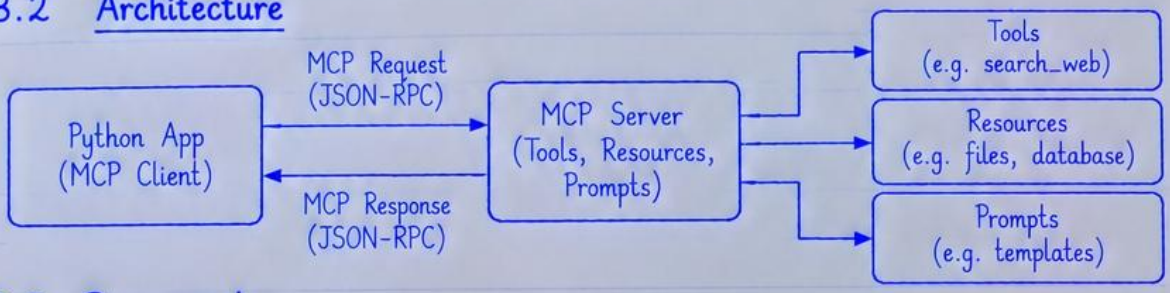
Rahul Yadav

8. Building a Simple MCP Client

8.1 Overview

- MCP clients can be built using existing SDKs or custom code.
- The client connects to an MCP server, discovers available tools and resources, sends requests and handles responses.
- We will build a simple Python MCP client to call a tool (e.g. search_web).
- This example uses standard JSON-RPC over HTTP (conceptual).

8.2 Architecture



8.3 Prerequisites

- Python 3.8+ installed.
- MCP client SDK (or use simple HTTP client).
- MCP server URL (e.g. http://localhost:8000/mcp).
- Valid API key or authentication (if required).

8.4 Example: Simple MCP Client in Python

```
import requests
import json

MCP_SERVER_URL = "http://localhost:8000/mcp"

def call_mcp_tool(tool_name, params):
    payload = {
        "jsonrpc": "2.0",
        "id": 1,
        "method": "tools/call",
        "params": {
            "name": tool_name,
            "arguments": params
        }
    }

    response = requests.post(
        MCP_SERVER_URL,
        headers={"Content-Type": "application/json"},
        json=payload
    )
    return response.json()

# Example usage
result = call_mcp_tool("search_web", {"query": "latest AI news"})
print(json.dumps(result, indent=2))
```

8.5 Explanation

1. We create a JSON-RPC request with method tools/call.
2. Provide the tool name and input parameters.
3. Send the request to the MCP server using HTTP POST.
4. Parse and return the JSON response.
5. Handle errors and authentication in production code.

8.6 Sample Response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Here are the latest
            AI news ...."
      }
    ]
  }
}
```

8.7 Key Points

- Use official SDKs if available (e.g. mcp, langchain-mcp, etc.).
- Handle authentication, timeouts and errors properly.
- You can call different tools like search, read_file, query_database, etc.
- The same client can be used with multiple MCP servers.

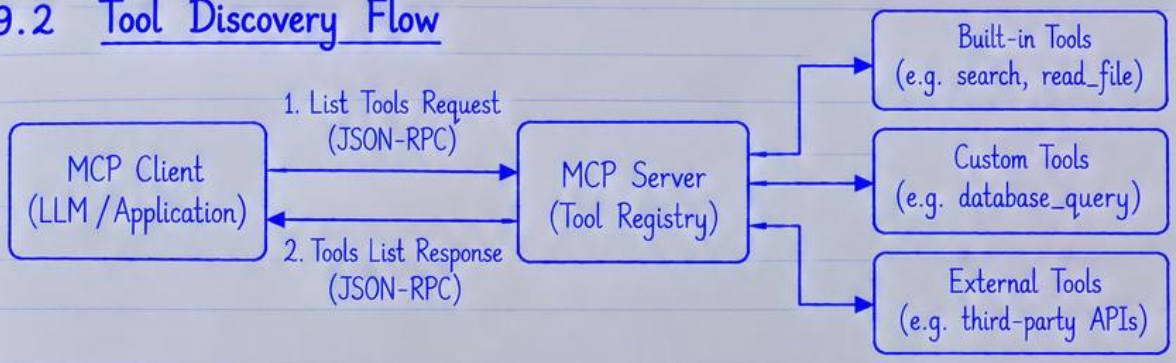
Rahul Yadav

9. Advanced MCP Features

9.1 Dynamic Tools and Resources

- MCP servers can expose dynamic tools based on context.
- Tools and resources can be added, updated or removed at runtime.
- This allows flexible integration with databases, APIs, and custom services.
- Clients can list available tools/resources before making a request.

9.2 Tool Discovery Flow



9.3 Example: Listing Tools (JSON-RPC)

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/list",
    "params": {}
}
```

9.4 Example Response

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
        "tools": [
            {
                "name": "search_web",
                "description": "Search the web for information",
                "input_schema": { ... }
            },
            {
                "name": "read_file",
                "description": "Read file content",
                "input_schema": { ... }
            }
        ]
    }
}
```

9.5 Resource Types

Resource Type	Description	Example
File	Access to local or remote files	Read PDF, TXT, DOC
Database	Query and fetch data from databases	MySQL, PostgreSQL, MongoDB
API	Call external REST/GraphQL APIs	Weather API, GitHub API
Custom	Domain specific resources	Company internal systems

9.6 Key Points

- Use tools/list to discover available tools.
- Tools can be dynamic and context-aware.
- Each tool has a name, description and input schema.
- Resources can include files, databases, APIs and custom systems.
- This makes MCP highly extensible for real-world applications.

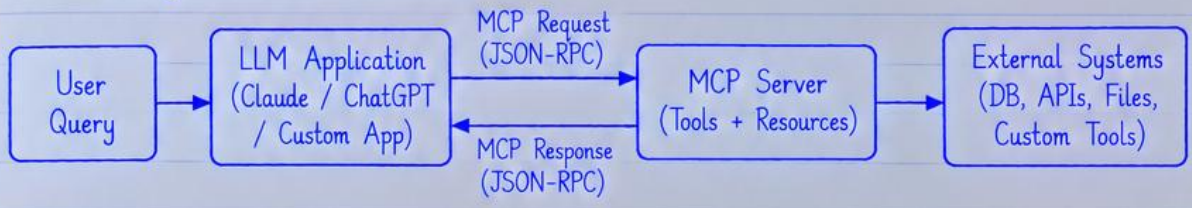
Rahul Yadav

10. Integrating MCP with Real Applications

10.1 Integration Approaches

- MCP can be integrated with LLM applications, backend services or custom tools.
- You can use official SDKs or simple HTTP client.
- Tools can be called dynamically based on user queries.
- Supports different programming languages (Python, TypeScript, Java, etc.).
- Can be integrated with frameworks like LangChain, LlamaIndex or custom solutions.

10.2 Integration Architecture



10.3 Example: Integration with LangChain (Conceptual)

```
from langchain_mcp import MCPClient

# Initialize MCP client
client = MCPClient(
    server_url="http://localhost:8000/mcp",
    api_key="your_api_key"
)

# Call a tool
result = client.call_tool(
    tool_name="search_web",
    params={"query": "latest AI news"}
)
print(result)
```

10.4 Example: Direct HTTP Integration

```
import requests
import json
url = "http://localhost:8000/mcp"
payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "read_file",
        "arguments": {"path": "/data/report.pdf"}
    }
}
response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)
print(response.json())
```

10.5 Real-World Integration Use Cases

Use Case	Description	Tools Used	Example
Document Search	Search internal documents	search_web, read_file	Company knowledge base
Database Query	Query business data	database_query	Customer or sales data
File Processing	Read and analyze files	read_file	PDF, DOC, TXT files
External API Calls	Access external services	custom tools	Weather, GitHub, CRM APIs
Automation	Trigger workflows	custom tools	Send email, create tickets

10.6 Key Points

- MCP can be integrated with any LLM or application.
- Use SDKs or HTTP client for integration.
- Tools can be called programmatically based on user intent.
- Common use cases include document search, database queries, file processing and automation.
- Makes LLM applications more powerful and useful in real-world scenarios.

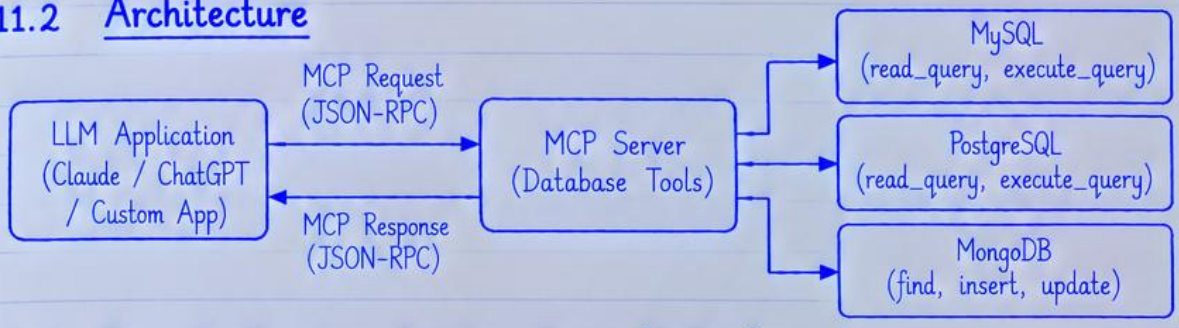
Rahul Yadav

11. MCP with Databases

11.1 Overview

- MCP can be used to connect with databases and run queries.
- The MCP server exposes database tools like read_query, execute_query, list_tables.
- This allows LLM applications to fetch and analyze real-time data from databases.
- Supports multiple databases like MySQL, PostgreSQL, MongoDB, etc.

11.2 Architecture



11.3 Example: Running a Database Query (Python)

```python
from mcp import ClientSession
from mcp.client.stdio import stdio_client
import asyncio

async def run_query():
    async with stdio_client() as (read, write):
        async with ClientSession(read, write) as session:
            # Call database query tool
            result = await session.call_tool(
                "read_query",
                {
                    "query": "SELECT id, name, email FROM users
                              LIMIT 5"
                }
            )
            print(result)

asyncio.run(run_query())
```

11.4 Sample Response

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
        "columns": ["id", "name", "email"],
        "rows": [
            [1, "Rahul", "rahul@example.com"],
            [2, "Preeti", "preeti@example.com"],
            [3, "Amit", "amit@example.com"]
        ]
    }
}
```

11.5 Other Useful Database Tools

- list_tables - Get all table names
- read_query - Run SELECT queries
- execute_query - Run INSERT/UPDATE/DELETE
- get_schema - Get table schema
- describe_table - Get column details

11.6 Supported Databases (Examples)

Database	Tools	Example Use Case
MySQL	read_query, execute_query, list_tables	Fetch customer data, run reports
PostgreSQL	read_query, execute_query, list_tables	Analytics, business data queries
MongoDB	find, insert, update, list_collections	Work with JSON documents
SQLite	read_query, execute_query	Local development and testing
Others	Custom tools via MCP server	Connect to any database

11.7 Key Points

- MCP enables secure and controlled access to databases.
- Use read-only queries for safety when possible.
- Validate and sanitize inputs to prevent SQL injection.
- You can create custom tools for specific database operations.
- Supports multiple databases based on your use case.

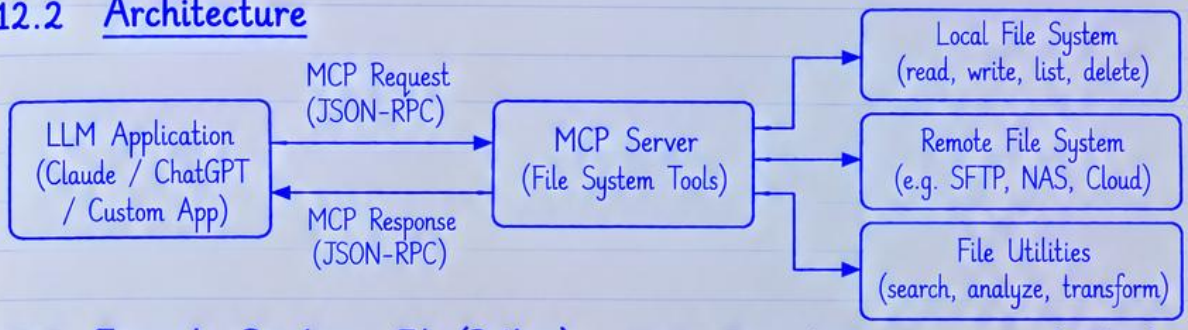
Rahul Yadav

12. MCP with File System

12.1 Overview

- MCP can be used to access and operate on local or remote file systems.
- The server exposes file system tools like read_file, write_file, list_directory, etc.
- This allows LLM applications to read, analyze, modify and manage files.
- Useful for document search, code analysis, log investigation and automation tasks.

12.2 Architecture



12.3 Example: Reading a File (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "read_file",
        "arguments": { "path": "./data/sample.txt" }
    }
}

response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)
print(response.json())
```

12.4 Example: Writing a File (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

payload = {
    "jsonrpc": "2.0",
    "id": 2,
    "method": "tools/call",
    "params": {
        "name": "write_file",
        "arguments": {
            "path": "./data/output.txt",
            "content": "Hello from MCP!\nThis file
            was created using MCP."
        }
    }
}
response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)
print(response.json())
```

12.5 Common File System Tools

Tool Name	Description	Example Input
read_file	Read content of a file	{ "path": "file.txt" }
write_file	Create or overwrite a file	{ "path": "file.txt", "content": "Hello" }
list_directory	List files in a directory	{ "path": "./data" }
delete_file	Delete a file	{ "path": "file.txt" }
search_files	Search files by pattern	{ "path": "./logs", "pattern": "*.log" }
get_file_info	Get file metadata	{ "path": "file.txt" }

12.6 Sample Response

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
        "path": "./data/sample.txt",
        "content": "Hello from MCP!\nThis
        is a sample file.\n",
        "size": 34,
        "modified": "2024-01-15T10:30:00Z"
    }
}
```

12.7 Key Points

- MCP provides secure and controlled access to file systems.
- Supports both local and remote file systems.
- Useful for document analysis, code base understanding and automation.
- File operations include read, write, list, delete, search and metadata.
- Can be integrated with LLM applications, agents and workflows.

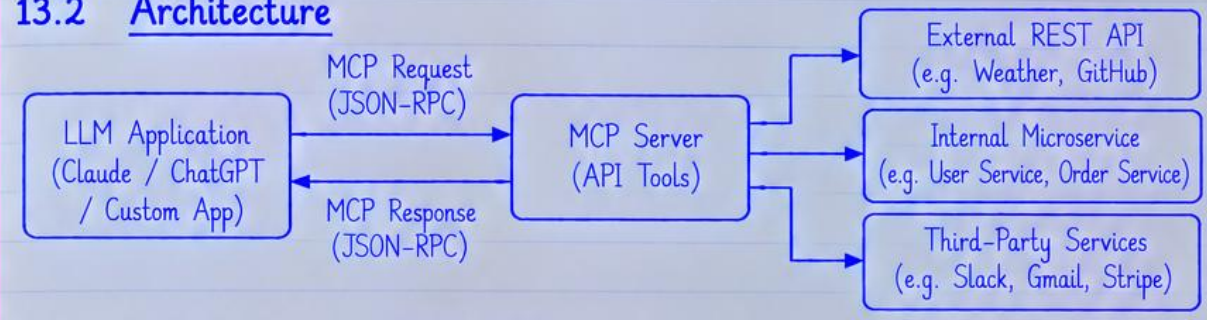
Rahul Yadav

13. MCP with APIs and External Services

13.1 Overview

- MCP can be used to call external APIs and third-party services.
- The MCP server acts as a bridge between LLM applications and external systems.
- Supports REST APIs, GraphQL APIs, internal microservices and third-party tools.
- Useful for real-world applications like weather data, payment systems, CRM, etc.

13.2 Architecture



13.3 Example: Calling a REST API (Python)

```python
import requests
import json

def call_weather_api(city: str):
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {
        "q": city,
        "appid": "your_api_key",
        "units": "metric"
    }

    response = requests.get(url, params=params)
    if response.status_code == 200:
        data = response.json()
        return {
            "temperature": data["main"]["temp"],
            "description": data["weather"][0]["description"],
            "humidity": data["main"]["humidity"]
        }
    else:
        return {"error": "Failed to fetch weather data"}
```

13.4 Example: MCP Tool for API Call

```python
from mcp.server.fastmcp import FastMCP
import requests

mcp = FastMCP("Weather API")

@mcp.tool()
def get_weather(city: str):
    """Get current weather information for a city."""
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": city, "appid": "your_api_key", "units": "metric"}

    response = requests.get(url, params=params)
    if response.status_code == 200:
        data = response.json()
        return {
            "city": city,
            "temperature": data["main"]["temp"],
            "description": data["weather"][0]["description"],
            "humidity": data["main"]["humidity"]
        }
    return {"error": "Failed to fetch weather data"}
```

13.5 Sample MCP Request

```json
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "get_weather",
        "arguments": {
            "city": "Jaipur"
        }
    }
}
```

13.6 Sample Response

```json
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
        "city": "Jaipur",
        "temperature": 32.5,
        "description": "clear sky",
        "humidity": 28
    }
}
```

13.7 Common External Services

Service	Use Case
REST API	Weather, News, Maps
GraphQL API	GitHub, Shopify, etc.
Internal API	User, Order, Product Service
Third-Party Tools	Slack, Gmail, Stripe
Cloud Services	AWS, GCP, Azure
Others	Custom business APIs

13.8 Key Points

- MCP can integrate with any external API or service.
- Use HTTP clients (like requests, httpx) inside tools.
- Handle authentication (API keys, OAuth) properly,
- Return structured JSON responses to the LLM.
- Useful for real-world applications like dashboards, automation and agent workflows.

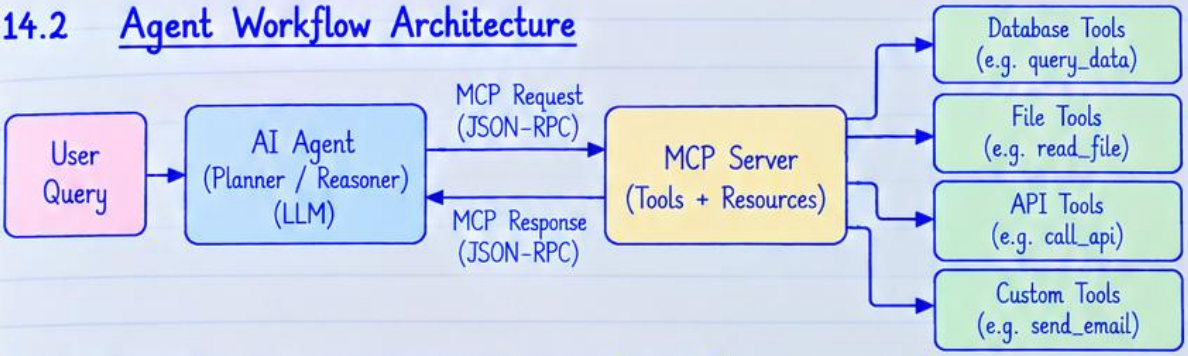
Rahul Yadav

14. MCP with Agents and Workflows

14.1 Overview

- MCP allows AI agents to discover and use external tools, resources and APIs.
- Agents can plan, decide and call multiple tools in a workflow.
- MCP provides a standard interface for tool discovery and execution.
- Useful for building autonomous agents, assistants and multi-step workflows.

14.2 Agent Workflow Architecture



14.3 Example: Agent with Multiple Tools (Python)

```python
from langchain_mcp import MCPClient
from langchain.agents import initialize_agent, Tool
from langchain_openai import ChatOpenAI

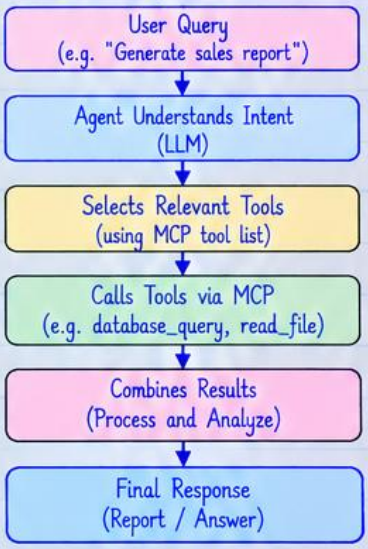
# Initialize MCP client
mcp_client = MCPClient(
    server_url="http://localhost:8000/mcp",
    api_key="your-api-key"
)

# Get available tools from MCP
mcp_tools = mcp_client.list_tools()

# Create an agent with MCP tools
llm = ChatOpenAI(model="gpt-4o")
agent = initialize_agent(
    tools=mcp_tools,
    llm=llm,
    agent_type="openai-tools",
    verbose=True
)

# Run a query
response = agent.run("Find latest sales data from database \
            and create a summary report")
print(response)
```

14.4 Example: Agent Decision Flow



14.5 Sample Agent Query and Response

User Query (JSON)

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "agent/run",
  "params": {
    "query": "Get top 5 customers
     from database and create a\
     summary report".
  }
}
```

Agent Response (JSON)

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "summary": "Top 5 customers fetched\
    and report generated.",
    "data": [
      {"customer": "ABC Ltd", "revenue": 120000},
      {"customer": "XYZ Inc", "revenue":98000}
    ],
    "report_file": "/reports/top_customers.pdf"
  }
}
```

14.6 Common Agent Use Cases

Use Case	Description
Data Analysis	Query databases and generate reports
Document Processing	Read and analyze files
Automation	Trigger workflows and APIs
Customer Support	Fetch information and respond
Content Generation	Use multiple tools to create content
Business Insights	Combine data from multiple sources

14.7 Key Points

- Agents can discover and use tools via MCP.
- Use LLM to decide which tools to call.
- MCP enables multi-step workflows and automation.
- Can integrate with databases, file systems, APIs and custom tools.
- Useful for building autonomous AI agents for real-world applications.

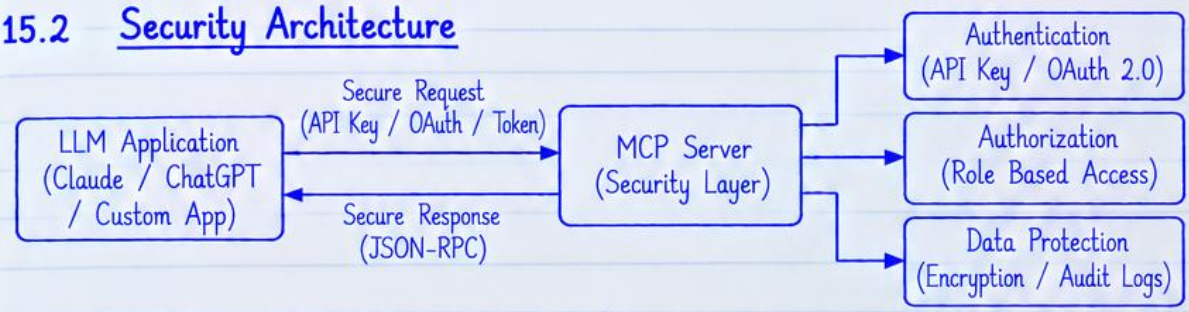
Rahul Yadav

15. MCP Security, Authentication and Deployment

15.1 Overview

- Security is important when connecting LLMs with external tools and data.
- MCP supports secure communication, authentication and access control.
- You can protect sensitive data using API keys, OAuth, tokens and permissions.
- Agents and applications should follow best practices for safe and reliable usage.

15.2 Security Architecture



15.3 Example: Using API Key Authentication (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

headers = {
    "Content-Type": "application/json",
    "Authorization": "Bearer YOUR_API_KEY"
}

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "read_data",
        "arguments": {"path": "/secure/data.txt"}
    }
}

response = requests.post(url, headers=headers, json=payload)
print(response.json())
```

15.4 Example: Using OAuth 2.0 (Python)

```python
from requests_oauthlib import OAuth2Session

client_id = "YOUR_CLIENT_ID"
client_secret = "YOUR_CLIENT_SECRET"
auth_url = "https://auth.example.com/authorize"
token_url = "https://auth.example.com/token"

# Create OAuth2 session
oauth = OAuth2Session(client_id)

token = oauth.fetch_token(
    token_url=token_url,
    client_secret=client_secret,
    authorization_response="YOUR_CALLBACK_URL"
)

headers = {"Authorization": f"Bearer {token['access_token']}"}
payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/list"
}
response = oauth.post("http://localhost:8000/mcp",
                      json=payload, headers=headers)
print(response.json())
```

15.5 Access Control and Permissions

Permission Type	Description	Example
Read Access	Allow read-only operations	read_file, list_data
Write Access	Allow create or update	write_file, insert_data
Tool Access	Restrict specific tools	allow only database_query
Role Based	Access based on user role	admin, editor, viewer
Resource Level	Restrict by resource path	/data/reports/*

15.6 Best Practices

- Use API keys or OAuth for authentication.
- Enable role-based access control.
- Validate inputs to prevent misuse.
- Use encryption for sensitive data.
- Keep audit logs for monitoring.
- Restrict access to only required tools.
- Use rate limiting to prevent abuse.
- Regularly rotate API keys and tokens.
- Follow least privilege principle.

15.7 Deployment Options

Deployment Option	Description	Use Case
Local Deployment	Run MCP server on your machine	Development and testing
Docker Deployment	Run using Docker container	Easy setup and portability
Cloud Deployment	Deploy on AWS, GCP, Azure	Production use, scalable
Kubernetes Deployment	Run multiple instances with K8s	High availability and scaling
Serverless Deployment	Use serverless functions	Event-driven and cost efficient

15.8 Key Points

- MCP supports secure and controlled access to tools and data.
- Use API keys, OAuth 2.0 and role-based permissions.
- Follow best practices like encryption, input validation and audit logging.
- Can be deployed locally, on cloud, Docker or Kubernetes.
- Security is essential for real-world AI agent applications.

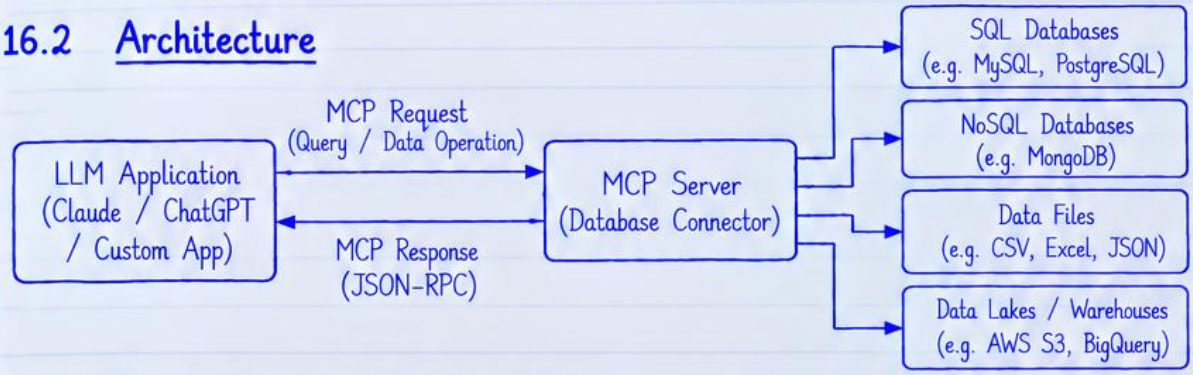
Rahul Yadav

16. MCP with Databases and Data Sources

16.1 Overview

- MCP can connect to various databases and data sources.
- It allows LLM applications to query, analyze and retrieve structured data.
- Supports SQL databases, NoSQL databases, data lakes and file-based sources.
- Useful for building data-driven AI agents and analytics applications.

16.2 Architecture



16.3 Example: Querying a Database (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "run_sql_query",
        "arguments": {
            "query": "SELECT * FROM customers \
                      WHERE city = 'Jaipur' LIMIT 5"
        }
    }
}

response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)

print(response.json())
```

16.4 Example: Querying MongoDB (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "mongodb_find",
        "arguments": {
            "collection": "users",
            "filter": {"city": "Jaipur"},
            "limit": 5
        }
    }
}

response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)

print(response.json())
```

16.5 Common Database Connectors

Database Type	Tool Name (Example)	Use Case
MySQL	mysql_query	Query relational data
PostgreSQL	postgres_query	Advanced SQL queries
MongoDB	mongodb_find	Query NoSQL documents
SQLite	sqlite_query	Local database files
CSV / Excel	read_csv / read_excel	Read tabular data
Data Lake (AWS S3)	s3_read	Read data from S3
BigQuery	bigquery_query	Analytics and reporting

16.6 Sample Response

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "columns": ["id", "name", "city", "age"],
    "rows": [
      {"id": 1, "name": "Rahul", "city": "Jaipur", "age": 28},
      {"id": 2, "name": "Amit", "city": "Delhi", "age": 32}
    ],
    "row_count": 2
  }
}
```

16.7 Key Points

- MCP can connect to multiple databases and data sources.
- Supports both structured (SQL) and unstructured (NoSQL, files) data.
- Use appropriate tools for each data source.
- Ensure proper authentication and access control.
- Useful for analytics, reporting and data-driven AI applications.

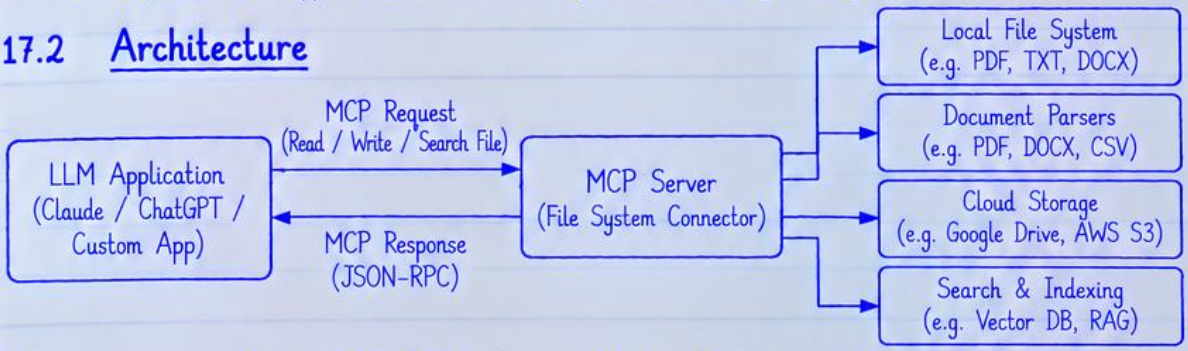
Rahul Yadav

17. MCP with File Systems and Document Tools

17.1 Overview

- MCP can work with file systems and document tools.
- It allows LLM applications to read, write, search and analyze documents.
- Supports local files, cloud storage (e.g. Google Drive, S3), and document parsers.
- Useful for RAG applications, document analysis and knowledge management.

17.2 Architecture



17.3 Example: Read a File (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "read_file",
        "arguments": {
            "path": "./data/notes.txt"
        }
    }
}
response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)
print(response.json())
```

17.4 Example: Write a File (Python)

```python
import requests
import json

url = "http://localhost:8000/mcp"

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "write_file",
        "arguments": {
            "path": "./data/output.txt",
            "content": "Hello from MCP!\nThis file
                       was created using MCP."
        }
    }
}
response = requests.post(
    url,
    headers={"Content-Type": "application/json"},
    json=payload
)
print(response.json())
```

17.5 Example: Use with Google Drive (Python)

```python
from langchain_mcp import MCPClient

mcp_client = MCPClient(
    server_url="http://localhost:8000/mcp",
    api_key="your-api-key"
)
# List files from Google Drive via MCP tool
response = mcp_client.run("list_drive_files", {
    "folder_id": "root",
    "limit": 10
})
print(response)
```

17.6 Common Document Tools

Tool / Service	Use Case	Supported Format
Local File System	Read / Write files	TXT, PDF, DOCX, CSV, JSON
PDF Parser	Extract text from PDF	PDF
Docx Parser	Extract text from Word	DOCX
CSV Reader	Read tabular data	CSV
Google Drive	Access cloud documents	All formats
AWS S3	Read / Write files	All formats
Vector DB (RAG)	Index and search documents	Text, PDF, DOCX

17.7 Use Cases

- Read internal documents and answer questions.
- Summarize PDF or Word files.
- Write reports, logs and analysis results.
- Integrate with Google Drive or AWS S3 for enterprise data.
- Build RAG applications over documents.

17.8 Key Points

- MCP provides standard tools for file and document operations.
- Supports local and cloud storage.
- Use appropriate parsers for different file formats.
- Can integrate with vector databases for RAG.
- Useful for enterprise knowledge management and AI applications.

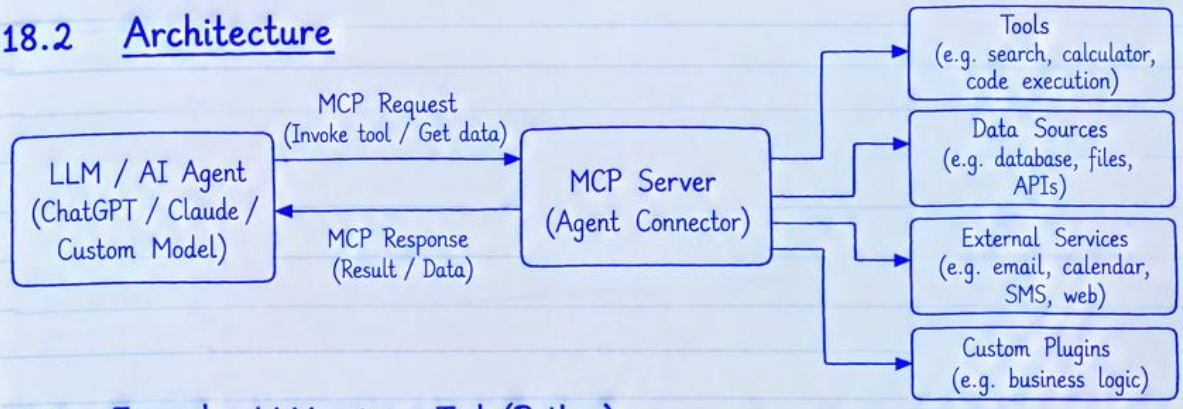
Rahul Yadav

18. MCP with LLMs and AI Agents

18.1 Overview

- MCP enables LLMs and AI agents to use external tools, data and services safely.
- It provides a standard way to connect LLMs with different capabilities.
- Supports function calling, tool use, and multi-step reasoning.
- Helps build autonomous AI agents with real-world context and actions.

18.2 Architecture



18.3 Example: LLM using a Tool (Python)

```
from mcp.client import MCPClient
from mcp.tools import search_tool

# Initialize MCP client
client = MCPClient(
    server_url="http://localhost:8000/mcp",
    api_key="your-api-key"
)
# Connect and call tool
client.connect()

# Use search tool
result = client.call_tool(
    "search",
    {"query": "latest AI news"}
)
print("Tool Result:", result)
```

18.4 Example: Build an AI Agent with MCP

```
from mcp.client import MCPClient
from mcp.agent import Agent

# Initialize agent with tools
agent = Agent(
    model="gpt-4o",
    tools=["search", "code", "file_read"],
    server_url="http://localhost:8000/mcp"
)
# Run the agent with a task
response = agent.run(
    "Find the latest updates on AI and save a summary
     to a file."
)
print(response)
```

18.5 Supported LLMs and Agent Frameworks

LLM / Framework	Integration Method	Key Features
OpenAI (ChatGPT)	Function Calling + MCP	Tool use, reasoning
Claude (Anthropic)	MCP Tools	Long context, analysis
Google Gemini	MCP + Function Calling	Multimodal, search
LangChain	MCP Toolkit	Agent workflows
CrewAI	MCP Tools	Multi-agent systems
AutoGen	MCP Integration	Conversational agents

18.6 Practical Use Cases

- Build AI agents that can search the web, read files and analyze data.
- Automate document summarization and report generation.
- Integrate with databases and cloud services.
- Create multi-agent systems for complex tasks.
- Use external APIs for real-world actions (e.g. send email, book meeting).

18.7 Key Points

- MCP standardizes how LLMs and agents use external tools and data.
- Supports function calling, tool use and multi-step workflows.
- Works with multiple LLMs and agent frameworks.
- Enables secure and controlled access to external services.
- Helps build powerful, autonomous and real-world AI agents.

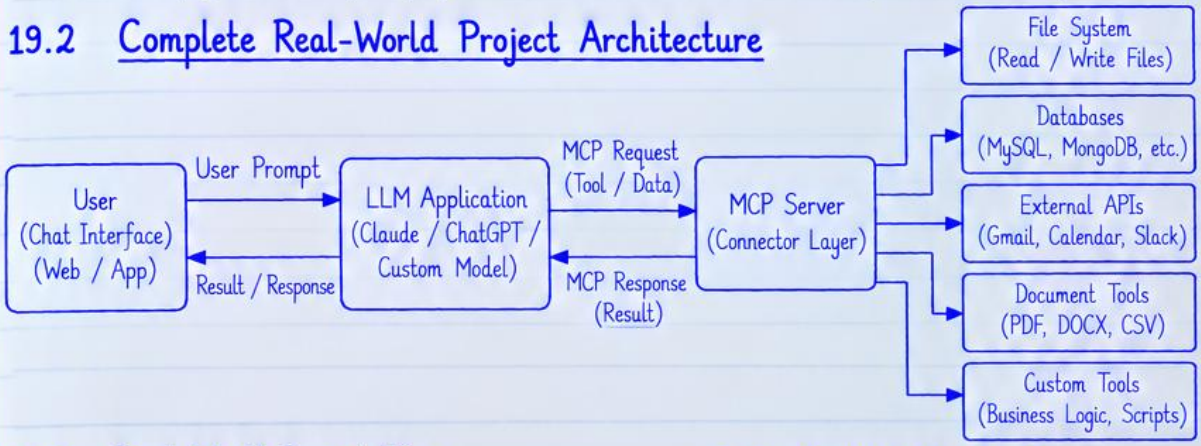
Rahul Yadav

19. Real-World Projects, Use Cases and Interview Preparation

19.1 Overview

- MCP can be used to build real-world applications by connecting LLMs with tools, databases, files, and external services.
- It enables AI agents to perform practical tasks like data analysis, document processing, automation and workflow orchestration.
- This page covers complete project ideas, real-world use cases and important interview questions.

19.2 Complete Real-World Project Architecture



19.3 Real-World Project Ideas

Project Name	Description	Tools Used
AI Document QA	Upload PDF/Docs and ask questions	MCP + PDF Parser + LLM
Data Analytics Agent	Query databases and generate reports	MCP + MySQL + Python
Personal Assistant	Read emails, create calendar events	MCP + Gmail API + Calendar API
File Management Agent	Search, read and organize local or cloud files	MCP + File System + LLM
Research Agent	Search web, collect data and create summary	MCP + Web Search + LLM
RAG Knowledge Base	Index documents and answer questions	MCP + Vector DB (Qdrant/FAISS)
Multi-Agent System	Use multiple agents for complex workflows	MCP + LangChain + Custom Tools

19.4 Real-World Use Cases

- Enterprise document search and analysis.
- Automated report generation from databases.
- Email and calendar management.
- Customer support automation.
- Knowledge base and internal documentation.
- Financial data analysis and insights.
- HR and employee information queries.
- Code analysis and repository search.
- Integration with business tools (Slack, Jira, Confluence).
- Build AI agents for repetitive tasks.

19.5 Interview Questions

1. What is MCP and why is it important?
2. How does MCP enable tool use for LLMs?
3. Explain the MCP architecture.
4. How does MCP handle authentication?
5. How will you connect MCP to a database?
6. How will you build a custom tool for MCP?
7. Explain the difference between function calling and MCP tools.
8. How can you build a multi-agent system using MCP?
9. What are the security considerations in MCP?
10. What are some real-world use cases of MCP?

19.6 Tips for Interview Preparation

- Understand the core concepts and architecture of MCP.
- Be comfortable building custom tools and integrations.
- Practice connecting MCP with databases, file systems and APIs.
- Work on at least one end-to-end project.
- Understand security, authentication and access control.
- Be ready to explain real-world use cases and trade-offs.
- Know related technologies like LangChain, OpenAI, vector databases and cloud services.

19.7 Key Takeaways

- MCP is a powerful standard for connecting LLMs with external tools and data sources.
- It can be used to build real-world AI applications and autonomous agents.
- Supports a wide range of use cases from document analysis to data analytics and automation.
- Understanding MCP is valuable for AI/ML engineers and software developers.
- Practice projects and interview preparation will help you build strong practical skills.

Rahul Yadav